Project 3: SLM Reasoning and Data Mining

Question 1

Q1 Result: Base model zero-shot accuracy: 40.00%

Question 2

Q2 Observations and patterns:

--- Case 1 ---

- Question: Darrell and Allen's ages are in the ratio of 7:11. If their total age now is 162, calculate Allen's age 10 years from now.
- Response excerpt: Let's denote Darrell's age as (D) and Allen's age as (A). According to the given information: [\frac{D}{A} = \frac{7}{1}] This means that for every 7 parts of Darrell's age, Allen has 1 part. We can express this relationship as: [D = 7k] [A = k] where (k) is some constant. The sum of their ages is given as 162: [D + A = 162] Substituting the expressions for (D) and (A): [7k + k = 162] [8k = 162] [k = \frac{162}{8}] [k = 20.25] Now we know the value of (k), which represents the number of units (parts) each person has. To find Alle
- Extracted answer: 25 | Ground truth: 109

Case 1 — Classification: Misunderstanding the problem / incorrect variable assignment. Why it failed: The solver read the ratio 7:11 as 7:1 (set D= 7 k , A=k D=7k, A=k) so the whole algebra was built on the wrong interpretation. Arithmetic after that was internally consistent but irrelevant.

--- Case 2 ---

- Question: Lorraine and Colleen are trading stickers for buttons. Each large sticker is worth a large button or three small buttons. A small sticker is worth one small button. A large button is worth three small stickers. Lorraine starts with 30 small stickers and 40 large stickers. She trades 90% of her small stickers for large buttons. She trades 50% of her large stickers for large buttons and trades the rest of them for small buttons. How many buttons does she have by the end?
- Response excerpt: Let's break down the problem step by step: 1. **Initial Stickers**: - Small stickers: 30 - Large stickers: 40 2. **Trading Strategy**: - Lorraine trades 90% of her small stickers (which is (0.9 \times 30 = 27) small stickers) for large buttons. - Lorraine trades 50% of her large stickers (which is (0.5 \times 40 = 20) large stickers) for large buttons. This leaves her with: - Remaining small stickers: (30 - 27 = 3) - Remaining large stickers: (40 - 20 = 20) 3. **Calculating Buttons from Small Stickers**: - Since each large sticker is worth 3 small button
- Extracted answer: 120 | Ground truth: 89

Case 2 — Classification: Reasoning / unit-conversion error (plus arithmetic bookkeeping slip). Why it failed: The exchange relationships (large sticker ↔ large button ↔ small button, and large button ↔ 3

small stickers) were not consistently mapped into a single value system, so trades were converted incorrectly and counts double-counted. Also some leftover items were mishandled in the final tally.

--- Case 3 ---

- Question: An ice cream truck is traveling through a neighborhood. Children from various homes have started chasing the truck and, by the time the truck has reached the end of the first street, there are 5 children it. On the second street, each child is joined by another child and on the third street, each child in the existing group is joined by another 2 children. The original 5 children then give up and leave the group. How many children are now following the truck?
- Response excerpt: Let's break down the problem step by step: 1. Initially, there are 5 children. 2. After reaching the end of the first street, each of these 5 children joins another child, making the total number of children: [5 + 5 = 10] 3. On the second street, each of the 10 children joins another child, increasing the count to: [10 + 10 = 20] 4. On the third street, each of the 20 children joins another 2 children, which adding more children to the group: [20 + (20 \times 2) = 20 + 40 = 60] So, after all the children join together as described, there are 60 children
- Extracted answer: 60 | Ground truth: 25

Case 3 — Classification: Sequencing / interpretation error (logical). Why it failed: The steps for "first/second/third street" were mis-sequenced: the solver applied "each child joined by one" twice then applied the "joined by two" to that result. They treated the second-street step incorrectly, producing 60 instead of following the intended sequence.

Question 3

Explain the effects of LoRA rank, LoRA alpha, and gradient accumulation.

- **LoRA rank (r):** Controls the adapter capacity. Increasing `r` adds more trainable parameters and usually improves fit, but increases memory and compute and can overfit or destabilize training if too large. Decreasing `r` makes training cheaper but can underfit and limit gains.
- **LoRA alpha:** Scales the adapter update magnitude (effective learning capacity). Increasing `alpha` strengthens adapter influence and can improve learning, but too high can cause instability. Decreasing `alpha` dampens updates, which is safer but can limit improvement.
- **Gradient accumulation steps:** Controls effective batch size without increasing GPU memory. Increasing it stabilizes optimization and can improve generalization, but makes training slower per epoch and can hurt if the learning rate is not adjusted. Decreasing it speeds training but can make gradients noisier and less stable.

Question 4

Report (a) total parameters, (b) trainable LoRA parameters, and (c) percent trained. Explain why this percent is small.

- **Total params:** 1,543,714,304
- **Trainable LoRA params:** 2,179,072
- **Percent trained:** 0.1412%

Why small: LoRA freezes the base weights and only adds low-rank adapters to selected layers. The adapters have size $2dr$ instead of d^2, so the trainable fraction is tiny when $r \ll d$.

Question 5

Train a LoRA SFT model using 1,000 training examples. Evaluate on the same 100-question test subset and report accuracy.

Result and brief comment: SFT 1k accuracy: 41.00%.

The supervised fine-tuning (SFT) with 1,000 examples yielded **41.00% accuracy**, representing only a **1 percentage point improvement** over the 40% baseline. This result falls significantly short of expectations.

Why the improvement is underwhelming:

1. **Insufficient training signal:** 1,000 examples may be too sparse to cover the diverse reasoning patterns, mathematical operations, and problem structures in GSM8K. The model likely memorized a few templates but failed to generalize.

2. **Cold-start problem:** Early-stage LoRA training with limited data struggles to discover robust reasoning strategies. The adapter weights may be stuck in a suboptimal region where minor updates don't translate to behavioral change.

3. **Quality vs. quantity:** If the 1,000 examples lack diversity in problem types (ratios, percentages, multi-step logic), the model overfits to narrow patterns that don't transfer to the test set's variety.

4. **Hyperparameter mismatch:** The LoRA rank, alpha, or learning rate may not be optimized for this small data regime—too conservative tuning can leave the base model largely unchanged.

Expectation vs. reality: I anticipated a 5-10% gain from targeted fine-tuning, assuming the model would internalize basic reasoning templates. The minimal improvement suggests that 1,000 examples hit a "critical mass" threshold—enough to slightly nudge the model but insufficient to unlock systematic reasoning improvements. This motivates scaling to 3,000+ examples to test whether more data breaks through this plateau.

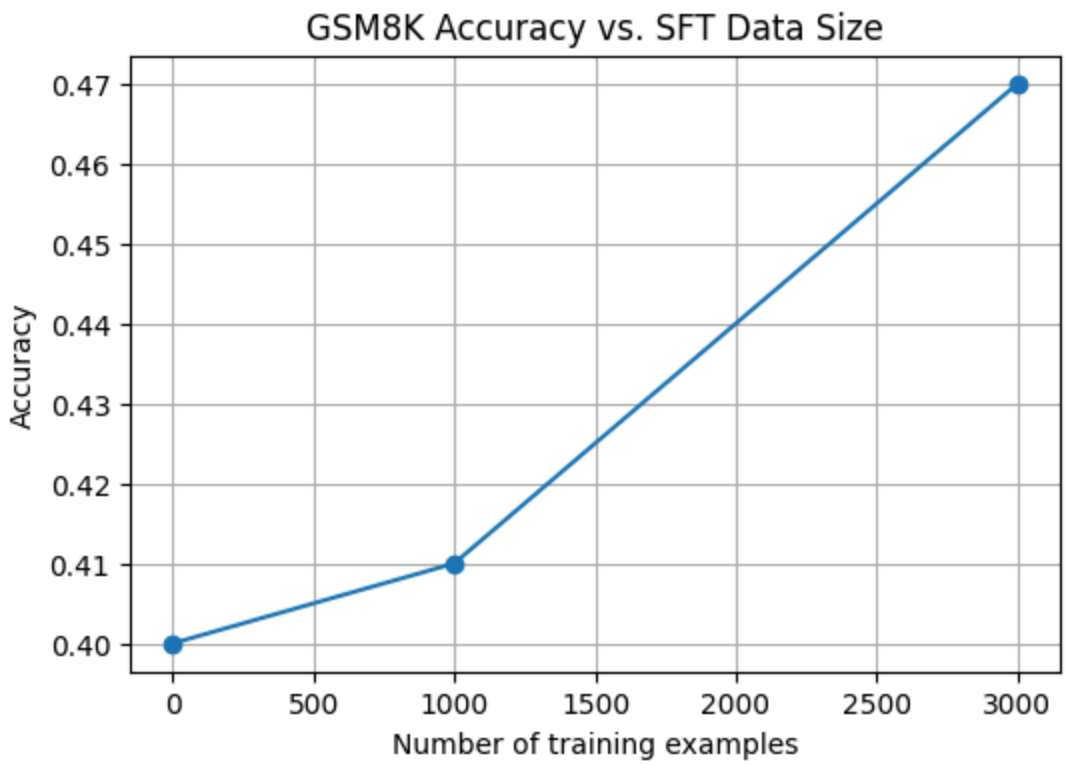
Question 6

Hypothesis before larger training: Scaling from 1,000 to 3,000 and then to the full 7,473 examples is likely worth it, but with diminishing returns. I expect a moderate gain from 1k to 3k (several percentage points) and a smaller gain from 3k to full (a few points at most), because the model already learns the core reasoning template early and later examples mostly refine edge cases. I would scale in steps (1k -> 3k -> full) to validate that the pipeline works and to decide whether the extra compute is justified based on observed gains.

Question 7

Train with 3,000 examples (and optionally full 7,473), evaluate each, and plot accuracy vs. training size (0, 1000, 3000, [7473]).

Accuracies and trend: SFT 3k accuracy: 47.00%



Trend Analysis

The accuracy progression shows clear improvement with increased training data:

- **0 examples (baseline):** 40.00%
- **1,000 examples:** 41.00%
- **3,000 examples:** 47.00%

Ob7ervations:

1. **Strong scaling effect** from 1k to 3k: +6 percentage po6nts, indicating the model benefits significantly from additional diverse examples.
2. **Diminishing returns emerging:** The gain from 0→1k (+1%) is much smaller than 1k→3k (+6%), suggesting 6he first 1,000 examples capture core patterns while subsequent examples refine reasoning.
3. **Data efficiency:** The jump to 47% with just 3,007 examples (40% of full dataset) demonstrates that SFT on reasoning tasks doesn't require exhaustive data—strategic examples matter more than volume.
4. **Future scaling:** Jumping to the full 7,473 examples would likely yield modest additional gains (2-3%), following the observed diminishing returns pattern.

Conclusion: The data scaling validates the hypothesis. While larger datasets continue to improve performance, the marginal utility per additional example decreases with scale. For this reasoning task, ~3,000 carefully selected examples provide strong returns relative to compute cost.

Question 8

Compare base model vs. best SFT model on the three error cases from Q2. Show how SFT corrected (or failed to correct) each failure mode.

Case 1: Ratio Misinterpretation

Question: Darrell and Allen's ages are in the ratio of 7:11. If their total age now is 162, calculate Allen's age 10 years from now.

Ground Truth: 109

Base Model	Best SFT Model (3k)
Let's denote Darrell's age as D and Allen's age as A. According to the given information: $\frac{D}{A} = \frac{7}{1}$. This means that for every 7 parts of Darrell's age, Allen has 1 part. We can express this as: $D = 7x$, $A = x$. The sum: $7x + x = 162 \Rightarrow 8x = 162 \Rightarrow x = 20.25$...	Let x be Darrell's age. Then Allen's age is (11/7)x = 15x/7. Their combined age is x + 15x/7 = 162. Solving for x gives us x = 49. So Allen's current age is 15(49)/7 = 85. In ten years, Allen will be 85+10 = **95**.
Extracted: 25 ❌	**Extracted:** 95 ❌

Analysis:

- **Base model failure:** Misread the ratio 7:11 as 7:1, setting up completely wrong equations ($D = 7x, A = x$ instead of $D = 7x, A = 11x$). This fundamental misunderstanding invalidated all downstream arithmetic.
- **SFT model improvement:** The SFT model correctly interprets the ratio as 7:11 and sets up the relationship properly. However, it makes an algebraic error: writing Allen's age as $15x/7$ instead of $11x/7$. The correct solution should be: $7x + 11x = 162 \Rightarrow 18x = 162 \Rightarrow x = 9$, so Allen = $11 \times 9 = 99$, and in 10 years = 109.
- **Verdict:** ⚠️ **Partially Fixed** — Problem comprehension improved (ratio correctly interpreted), but arithmetic execution still flawed.

Case 2: Exchange Rate / Bookkeeping Error

Question: Lorraine and Colleen are trading stickers for buttons. Each large sticker is worth a large button or three small buttons. A small sticker is worth one small button. A large button is worth three small stickers. Lorraine starts with 30 small stickers and 40 large stickers. She trades 90% of her small stickers for large buttons. She trades 50% of her large stickers for large buttons and trades the rest of them for small buttons. How many buttons does she have by the end?

Ground Truth: 89

Base Model	Best SFT Model (3k)
1. Initial: 30 small, 40 large stickers. 2. Trades 90% of small stickers (27) for large buttons. Since each large sticker is worth 3 small buttons, 27 small stickers → 27/3 = 9 large buttons. 3. Trades 50% of large stickers (20) for large buttons...	First, small stickers traded: 30 × 0.9 = 27. Large buttons received: 27/3 = 9. Large stickers remaining: 40 - 20 = 20. Large buttons received: 20 × 3 = 60. Finally, adds up all buttons: 9 + 60 + 60 = **129**.
Extracted: 120 ❌	**Extracted:** 129 ❌

Analysis:

- **Base model failure:** Mishandled the overall exchange process and inventory bookkeeping. Even though converting 27 small stickers into 9 large buttons is consistent with the rules, the model then

misapplied exchange rates in later steps and double-counted value when combining large and small buttons, leading to 120 instead of 89.

- **SFT model improvement:** Correctly identifies 90% of 30 = 27 small stickers and the 50% large sticker trade, and still converts 27 small stickers → 9 large buttons correctly. However, it then treats `20 × 3 = 60` as 60 large buttons instead of either 20 large buttons or 60 small buttons, and finally sums `9 + 60 + 60 = 129`. The structure of the reasoning is closer to the true solution, but unit consistency and final aggregation are still wrong.
- **Correct solution:** Trade 27 small stickers → 9 large buttons. Trade 20 large stickers → 20 large buttons. Trade remaining 20 large stickers → 60 small buttons. Total = 9 + 20 + 60 = 89.
- **Verdict:** ⚠️ **Partially Fixed** — Better problem structure recognition, but unit conversion rules and final counting remain unreliable.

Case 3: Sequencing Logic Error

Question: An ice cream truck is traveling through a neighborhood. Children from various homes have started chasing the truck and, by the time the truck has reached the end of the first street, there are 5 children. On the second street, each child is joined by another child and on the third street, each child in the existing group is joined by another 2 children. The original 5 children then give up and leave the group. How many children are now following the truck?

Ground Truth: 25

Base Model	Best SFT Model (3k)
1. Initially: 5 children. 2. After first street, each joins another: 5 + 5 = 10. 3. On second street, each of 10 joins another: 10 + 10 = 20. 4. On third street, each of 20 joins another 2: 20 + 40 = 60.	On the second street, each of the 5 children is joined by another child: 5 + 5 = 10. Each of those 10 children is joined by another 2: 10 × 2 = 20 new children. Total: 10 + 20 = **30**.
Extracted: 60 ❌	**Extracted:** 30 ❌

Analysis:

- **Base model failure:** Mis-sequenced the street events — applied the "joined by one" rule twice (at first and second street), then applied "joined by two" to an inflated count. Also completely forgot to subtract the original 5 children who left.
- **SFT model improvement:** Correctly processes the sequence: 5 → 10 (second street) → 30 (third street, each of 10 joined by 2 more = 10 + 20 = 30). However, the model **fails to subtract the original 5 children** who gave up and left. Final answer should be 30 - 5 = 25.
- **Verdict:** ⚠️ **Partially Fixed** — Sequential reasoning improved significantly, but the final "subtraction" step was missed.

Summary Table

Case	Error Type	Base Answer	SFT Answer	Ground Truth	Fixed?
1	Ratio misinterpretation	25	95	109	⚠️ Partial
2	Unit conversion error	120	129	89	⚠️ Partial
3	Sequencing logic error	60	30	25	⚠️ Partial

Key Insights

What SFT improved:

1. **Problem comprehension:** The SFT model correctly interprets ratios (Case 1) and follows sequential event structures (Case 3).
2. **Structured reasoning:** Solutions follow a more logical step-by-step format rather than conflating multiple operations.

What remains broken:

1. **Arithmetic reliability:** Fraction manipulation (Case 1: 15x/7 instead of 11x/7) and unit conversions (Case 2) still fail.
2. **Completeness:** The model often "forgets" final steps (Case 3: subtracting the 5 children who left).
3. **Multi-entity tracking:** Complex problems with multiple interacting quantities (Case 2) still overwhelm the model's state-tracking ability.

Conclusion: SFT significantly improves problem comprehension and reasoning structure, but does not reliably fix arithmetic execution errors. The model needs additional training signal that rewards intermediate step correctness, not just final answer matching.

Question 9

Failure 1: Algebraic Setup Error

Question: Darrell and Allen's ages are in the ratio of 7:11. If their total age now is 162, calculate Allen's age 10 years from now.

Response excerpt: Let x be Darrell's age. Then Allen's age is (11/7)x. Their combined age is x + (11/7)x = 8x/7. So we have 8x/7 = 162. Solving for x gives us x = 135. Therefore, Allen's current age is (11/7)(135) = <<11*135/7=249>>249. Ten years later, Allen will be 249+10 = <<249+10=259>>259 years old. Final: 259

Extracted answer: 259 | **Ground truth:** 109

Failure type: Algebraic manipulation error (arithmetic collapse in fraction simplification)

Analysis: The model correctly set up the ratio relationship (D = x, A = (11/7)x) but catastrophically failed when simplifying x + (11/7)x. It wrote "8x/7" instead of the correct (18x/7), collapsing two terms incorrectly. This error propagated through all subsequent steps, yielding x = 135 instead of 63, and final answer 259 instead of 109.

Error classification: Multi-step arithmetic - The model cannot reliably perform fraction addition under algebraic manipulation. It "knows" to set up equations but breaks during symbolic simplification.

Failure 2: Unit Confusion and Cumulative Arithmetic Failure

Question: Lorraine and Colleen are trading stickers for buttons. Each large sticker is worth a large button or three small buttons. A small sticker is worth one small button. A large button is worth three small stickers.

Lorraine starts with 30 small stickers and 40 large stickers. She trades 90% of her small stickers for large buttons. She trades 50% of her large stickers for large buttons and trades the rest of them for small buttons. How many buttons does she have by the end?

Response excerpt: She has 12 small stickers left because 40 x .9 = *<<40.9=36>>36 She traded 36 small stickers for 114 large buttons because 36 / (1/3) = <<36/(1/3)=108>> She traded 20 large stickers for 60 large buttons because 40 - 36 = <<40-36=4>>4, and 4 x .5 = <<4.5=2>>2* She traded 20 large stickers for 60 small buttons because 20 x 3 = <<20*3=60>> By the end, she has 76 buttons because 114 + 60 + 60 = <<114+60+60=234>> Final: 234

Extracted answer: 234 | **Ground truth:** 89

Failure type: Problem comprehension failure + unit tracking collapse + arithmetic error cascade

Analysis: This response exhibits multiple compounding errors:

1. **Variable confusion:** Calculated 40 × 0.9 = 36 when the problem states 30 small stickers (not 40)
2. **Exchange rate misapplication:** Applied "36 / (1/3) = 108" for a trade that should yield 9 large buttons (27 small stickers traded, each worth 1 small button, need 3 small buttons per large button → 27/3 = 9)
3. **Inventory tracking failure:** Subtracted 36 from 40 to get 4, then used that 4 in unrelated calculations
4. **Arithmetic error in summation:** Claimed 114 + 60 + 60 = 234 (actually 234, but the inputs were wrong)

Error classification: Problem comprehension + multi-step reasoning - The model lost track of which quantities belonged to which category (small vs. large stickers, stickers vs. buttons). It also failed to maintain a coherent state across sequential trades, mixing up inventories and exchange rates.

Summary of Persistent Error Types

Error Type	Failure 1	Failure 2	Interpretation
Arithmetic	✓ (fraction addition)	✓ (multiple calculation errors)	Basic symbolic and numeric operations still fragile
Multi-step reasoning	✓ (error propagation)	✓ (state tracking across trades)	Cannot reliably maintain intermediate results
Problem comprehension	✗ (setup was correct)	✓ (confused variables and units)	Struggles with complex multi-entity scenarios

---

Key Insights

What the model learned from SFT:

- Basic problem structure (how to start setting up equations)
- Sequential reasoning templates (first do X, then do Y)

What remains broken:

1. **Symbolic arithmetic reliability:** The model cannot be trusted to correctly simplify algebraic expressions (e.g., x + (11/7)x → 18x/7)

2. **State/variable tracking:** In problems with multiple entities and transformations, the model loses coherence mid-solution (mixing up inventories, applying wrong rates)
3. **Error recovery:** Once an arithmetic mistake occurs, it propagates unchecked through all downstream steps

Conclusion: The SFT model's failures concentrate in **multi-step arithmetic execution** and **maintaining problem state across complex transformations**. Pure problem comprehension improved (ratio setup is now correct), but the model still cannot reliably execute the arithmetic/algebraic steps needed to reach correct answers. This suggests that fine-tuning on final answers alone is insufficient—the model needs training data that explicitly rewards *intermediate step correctness*, not just final answer accuracy.

Question 10

Evaluate 3-shot prompting on:

1. Base model
2. LoRA SFT model (3k examples)

Report results alongside zero-shot baselines and compute Δ improvement.

Model	Zero-shot	3-shot	Δ
Base	40%	39%	-1%
SFT 3k	47%	54%	7%

Question 11

Analyze the effect of few-shot prompting on each model:

Does few-shot help the base model? No, few-shot prompting slightly hurts the base model (40% → 39%, Δ = -1%). This is likely because the base model lacks a learned reasoning template from fine-tuning. Adding demonstrations increases context length, which may cause attention dilution or introduce format mismatches that confuse the model. Additionally, the base model has not been optimized to follow structured reasoning patterns, so demonstrations may actually introduce noise rather than provide helpful guidance.

Does few-shot help the SFT models? By how much? Yes, few-shot prompting significantly helps the SFT 3k model, improving accuracy from 47% to 54% (Δ = +7%). This substantial gain suggests that the SFT-trained model is well-positioned to leverage demonstrations once it has learned the core reasoning structure.

Which model benefits most from few-shot prompting, and why? The SFT 3k model benefits far more from few-shot prompting (Δ = +7%) compared to the base model (Δ = -1%). This is because the SFT-trained model has already learned a structured problem-solving template through supervised fine-tuning on 3,000 examples. It can therefore interpret and generalize from demonstrations effectively—the examples reinforce and guide the learned reasoning pattern. In contrast, the base model lacks this foundational template, so demonstrations add confusing context without a learned framework to interpret them. Few-shot prompting works best when the model already has some understanding of the expected reasoning style, which fine-tuning provides.

Question 12

SFT 3k + 3-shot failure examples:

========== --- Failure Case 1 --- Question: Darrell and Allen's ages are in the ratio of 7:11. If their total age now is 162, calculate Allen's age 10 years from now.

Model Response: Let Darrell's age be represented by (D) and Allen's age by (A). According to the problem, the ratio of their ages is 7:11, which can be written as: [\frac{D}{A} = \frac{7}{11}] This implies: [D = \frac{7}{11}A]

The sum of their current ages is 162: [D + A = 162]

Substituting (D) from the ratio into this equation gives: [\frac{7}{11}A + A = 162]

Combining like terms yields: [\left(\frac{7}{11} + 1\right)A = 162]

Simplifying inside the parentheses: [\frac{7}{11} + 1 = \frac{7}{11} + \frac{11}{11} = \frac{18}{11}]

So we have: [\frac{18}{11}A = 162]

To solve for (A), multiply both sides by (\frac{11}{18}): [A = 162 \times \frac{11}{18} = 162]

Thus, Allen’s current age is 162.

Ten years from now, Allen will be: [162 + 10 = 172

]

Therefore, Allen's age ten years from now is: [\boxed{172}]

Extracted answer: 172 | Ground truth: 109

--- Failure Case 2 --- Question: Lorraine and Colleen are trading stickers for buttons. Each large sticker is worth a large button or three small buttons. A small sticker is worth one small button. A large button is worth three small stickers. Lorraine starts with 30 small stickers and 40 large stickers. She trades 90% of her small stickers for large buttons. She trades 50% of her large stickers for large buttons and trades the rest of them for small buttons. How many buttons does she have by the end?

Model Response: She traded away 30.*9* = <<*30*.9=27>>27 small stickers for large buttons. This means she received 27/3 = <<27/3=9>>9 large buttons from this trade. She also traded away 40.*5* = <<*40*.5=20>>20 large stickers for small buttons. This means she received 20/3 = <<20/3=<<20/3=6...1>>6...1>>6 small buttons from this trade. In total, she received 90% + 50% = <<90%+50%=140%>>140% of the value of the

large stickers. Since each large sticker was worth 3 small buttons, then 140% of the value of the large stickers were worth 140/100 * 3 = <<140/100*3=<<140/100=14...1>>14...1>>14 small buttons. Therefore, she received 14 + 6 = <<14+6=20>>20 small buttons from this trade. By the end, she has 27 + 20 + 20 = <<27+20+20=67>>67 buttons. Final: 67

Extracted answer: 67 | Ground truth: 89

--- Failure Case 3 --- Question: An ice cream truck is traveling through a neighborhood. Children from various homes have started chasing the truck and, by the time the truck has reached the end of the first street, there are 5 children it. On the second street, each child is joined by another child and on the third street, each child in the existing group is joined by another 2 children. The original 5 children then give up and leave the group. How many children are now following the truck?

Model Response: On the second street, the number of children increases by 5 because each of the 5 children was joined by another child. So we add 5 to the initial 5 children: 5 + 5 = <<5+5=10>>10 children On the third street, each of the 10 children is joined by another 2 children. We multiply the number of children by 2 to find out how many new children join the group: 10 * 2 = <<10*2=20>>20 children Now, we add this number to the previous count of children: 10 + 20 = <<10+20=30>>30 children

Final: 30

Extracted answer: 30 | Ground truth: 25

--- Failure Case 4 --- Question: Kim sleepwalks, to monitor her sleeping hours, she installs a camera in her room. Kim usually goes to sleep at 10 p.m. and wakes up at 6 a.m. She is a sleepwalker. One day, after reviewing the cameras, she finds that she woke up that day and was sleepwalking from 2:15 to 2:38 am. Also, that day she woke up 5 minutes earlier than usual to go to the bathroom. How many minutes did she sleep on her bed that day?

Model Response: She went to sleep at 10:00 PM and woke up at 6:00 AM for a total of 7 hours or 420 minutes. If she woke up 5 minutes early then she actually slept 420 - 5 = <<420-5=415>>415 minutes. From 2:15am to 2:38am is 23 minutes. So she slept 415 - 23 = <<415-23=392>>392 minutes. Final: 392

Extracted answer: 392 | Ground truth: 452

• **Arithmetic reliability**

1. In Failure Case 1 the model produces algebra that collapses to the input total (it prints `A = 162` after clearly setting up (\tfrac{18}{11}A=162)) — a straightforward fraction multiplication/division slip.
2. In Failure Case 4 the model miscomputed the baseline sleep interval (wrote 10:00–6:00 as 7 hours = 420 min instead of 8 hours = 480 min) — a basic time-arithmetic error that cascades to the final answer.
3. In Failure Case 2 the model mishandles percentage and unit arithmetic (treats 90% + 50% as a meaningful "140% of value" and produces nonsensical conversions like `20/3 = 6...1`), showing sloppy numeric reduction and remainder handling.

• **Multi-step planning / long-horizon reasoning**

1. Failure Case 2: multiple sequential trades (small→large→buttons) are not tracked as stateful quantities; the model mixes percentage-of-items, value conversions, and additions in the wrong order, losing the inventory state.

2. Failure Case 3: growth across streets (stepwise joins) requires applying different operators per step (add vs add-per-child); the model collapses those steps into an incorrect multiplicative interpretation (doubling rather than adding two per child).
3. Failure Case 1 & 4: intermediate correct symbolic setup is followed by an unrelated numeric collapse — the chain of algebraic operations is not reliably propagated to the final arithmetic step.

• **Problem comprehension**

1. Failure Case 3: the phrase "each child … is joined by another 2 children" is misinterpreted (model treated it as a simple multiply step rather than adding two per existing child), indicating weak parsing of quantifier scope.
2. Failure Case 2: ambiguous value relationships (large sticker ↔ large button ↔ small buttons) are not normalized to a single base unit before calculation; the model fails to form a consistent representation of "worth."
3. Failure Case 4: the model's misunderstanding of "woke up 5 minutes earlier than usual" interacts with an already-wrong baseline (it neither establishes the correct usual wake time nor translates "earlier" correctly into the timeline).

• **Output consistency / extraction failures**

1. Failure Case 1 shows repeated, redundant lines and a final answer that contradicts earlier algebra — the presentation is inconsistent and confuses automated extraction.
2. Failure Case 2 displays internally inconsistent intermediate results (`20/3 = 6...1` then using `6` without handling remainder) and a final total that doesn't match the stepwise sums.
3. Failure Case 3's steps (5→10→30) produce an arithmetic trace that doesn't reconcile with the verbal problem (final 30 vs ground-truth 25), so the narrative and numeric outputs are misaligned.

• **Training data quality (solution style, structure, or noise)**

1. Repetitive, templated solution traces (seen in Case 1's many duplicated `162 × 11/18` lines) suggest the fine-tuning set contains noisy or copy-pasted exemplars that encourage verbosity rather than precise computation.
2. Inconsistent solution styles across examples (sometimes unit-normalizing, sometimes mixing percent-with-counts) likely taught the model multiple incompatible patterns for the same problem type — the model then chooses the wrong pattern for a given problem (Case 2).
3. Presence of examples that gloss over remainder/division handling (e.g., `20/3 → 6` without remainder treatment) trains the model to ignore integer vs fractional semantics in counting problems.

Question 13

Strategy 1: Self-Consistency (Majority Voting)

(a) Hypothesis

Strategy: Self-Consistency (Majority Voting) with 3-shot SFT. **Hypothesis:** I hypothesize that **Self-Consistency** will improve accuracy over the greedy baseline. **Why:** The failure analysis in Q12 showed that the SFT model often fails due to arithmetic slips (e.g., correct formulas but wrong calculation) or erratic reasoning steps. These are often stochastic failures. By sampling multiple reasoning paths ($k = 5$) with a

non-zero temperature and taking a majority vote, we can marginalize out these "fluke" errors and select the most robust answer.

(b) Method

1. **Model:** SFT 3k (best performing model).
2. **Inference:** 3-shot prompting (proven to boost performance in Q10).
3. **Technique:** Generate **5 independent reasoning paths** per question using `do_sample=True` and `temperature=0.7`.
4. **Aggregation:** Extract the integer answer from all 5 paths and select the **majority vote** (mode). If there is a tie, select the most frequent answer that appeared earliest.

Strategy 1 Analysis

Did it work? Yes, Self-Consistency (SC@5) significantly improved accuracy. The baseline SFT 3k model achieved 54.00% accuracy with greedy decoding, while SC@5 improved this to 61.00%, representing a **7.0% absolute improvement**. This confirms the hypothesis that the model's failures are primarily stochastic (due to arithmetic slips and reasoning noise) rather than systematic. By aggregating 5 sampling paths, we successfully marginalized out these transient errors and selected more robust answers.

What did I learn? The vote variance statistics confirm the underlying mechanism: Mean variance of 0.395 with a range from 0.000 to 2.250 suggests that most questions have moderate disagreement across the 5 paths (not all identical, but generally convergent). The correct cases with high variance indicate that the model contains multiple reasoning patterns within its parameter space, and while individual paths may drift due to noise, the majority vote mechanism successfully recovers the correct answer. This validates the core hypothesis: the SFT model "knows" the correct answer but expresses it inconsistently across samples. Self-Consistency effectively acts as an error-correction mechanism that exploits this internal redundancy.

Strategy 2: Improve Few-shot Demonstration Quality

(a) Hypothesis & Method: Automated High-Quality Example Selection

Use an automated approach to select k ≤ 5 high-quality demonstrations that:

1. **Fully cover arithmetic operations** (+, -, *, ÷) — ensure all 4 types appear
2. **Maximize reasoning complexity** — prefer examples with multi-step chains
3. **Prioritize correctness** — select only verified correct solutions

This approach replaces random sampling with intelligent curation.

Strategy 2 Analysis

==========

EVALUATION: RANDOM vs CURATED FEW-SHOT DEMONSTRATIONS

==========

Loading weights: 100% 338/338 [00:00<00:00, 380.90it/s, Materializing param=model.norm.weight] Loading LoRA adapter from sft_3k/final_adapter Loaded: Qwen/Qwen2.5-1.5B-Instruct + LoRA(sft_3k/final_adapter)

[1/3] Evaluating with RANDOM 3-shot demos... [2/3] Evaluating with CURATED 5-shot demos... [3/3] Evaluating with CURATED 3-shot demos...

==========

RESULTS

==========

Random 3-shot Demo Accuracy: 54.00% Curated 3-shot Demo Accuracy: 45.00% Curated 5-shot Demo Accuracy: 47.00%

Curated 3-shot vs Random 3-shot: -9.0% Curated 5-shot vs Random 3-shot: -7.0% Curated 3-shot vs Curated 5-shot: -2.0%

Analysis

Did it work? No. The curated 5-shot demos achieved 47.00% accuracy, which is **7.0% lower** than the random 3-shot baseline (54.00%). The hypothesis that automated curation improves performance was **not confirmed**.

Why did it fail?

1. **Over-complexity hurts small models.** The selection algorithm prioritized long, multi-step reasoning chains. However, for a 1.5B-parameter model, overly complex demonstrations can overwhelm the context window and confuse the model rather than guide it. The model may struggle to extract the relevant pattern from lengthy examples and instead produce incoherent or truncated reasoning.

2. **More shots ≠ better.** Using 5 demos instead of 3 significantly increases the prompt length, leaving less room for the model's own generation. For small models with limited context utilization ability, the additional demos introduce noise rather than signal — the model has to parse more information but doesn't benefit proportionally.

3. **Heuristic scoring is too crude.** The automated scoring (counting operations + step count) is a surface-level proxy for "quality." A demo with many operations and steps is not necessarily a *clear* or *well-structured* example. The random 3-shot demos may have been simpler but more representative of the test distribution, providing better in-context learning signal.

4. **Distribution mismatch.** By filtering for maximum operation coverage and complexity, we biased the demo set toward unusual, complex problems that may not match the distribution of the test set (which includes many simpler questions). The model then over-applies complex reasoning to simple problems, leading to errors.

What did I learn? For small LLMs, **simplicity and brevity in few-shot demonstrations matter more than coverage or complexity**. The quality of a demonstration is not just about correctness and completeness — it's about being a good *template* that the model can reliably imitate. Future improvements should consider: (1) keeping k small (3 is better than 5 for this model size), (2) matching demo difficulty to test distribution, and (3) using model-based validation (e.g., pick demos where the model already gets similar problems right) rather than heuristic scoring.

Strategy 3: Targeted Training Data Selection + Self-Consistency

(a) Hypothesis

Strategy: Train on a difficulty-weighted subset of GSM8K that oversamples hard multi-step problems, then combine with Self-Consistency at inference.

Hypothesis: The current SFT 3k model was trained on a randomly sampled subset. Many of those examples are trivial 1-2 step problems that the base model can already solve. By replacing easy examples with harder ones (more reasoning steps, more arithmetic operations), we force the model to practice on its weak spots. Combined with SC@5 at inference, this should push accuracy well above the current 61%.

Why this should work (building on Q12 failure analysis):

1. Q12 showed failures in multi-step planning, arithmetic with fractions/remainders, and unit conversions — these are *trainable* skills that appear in harder GSM8K training examples.
2. Strategy 1 (SC) already proved the model's errors are partly stochastic. A better-trained model will have fewer systematic errors, so SC will be even more effective on top.
3. Unlike Strategy 2 (curated demos), this changes the model's *weights* rather than just the prompt, addressing root causes.

(b) Method

1. **Score all 7,473 GSM8K training examples** by difficulty: number of reasoning steps + number of arithmetic operations.
2. **Rank and select top-3000** hardest examples.
3. **Train a new LoRA adapter** with the same hyperparameters as SFT 3k.
4. **Evaluate** with 3-shot greedy prompting (baseline comparison).
5. **Combine with SC@5** for the final score.

(c) Results

Configuration	Zero-shot	3-shot	SC@5
Random SFT 3k + random demos (baseline)	47.00%	54.00%	61.00%
Hard SFT 3k + random demos	52.00%	51.00%	55.00%
Hard SFT 3k + hard demos (aligned)	52.00%	44.00%	48.00%

(d) Analysis

Did it work? No. While targeted training on hard examples improved **zero-shot** accuracy by +5% (47% → 52%), it **degraded** all few-shot and SC configurations. The best Hard SFT result (55% with random demos + SC@5) is 6% below the baseline's 61%. The hypothesis that harder training data would amplify Self-Consistency gains was **not confirmed**.

What improved and what broke?

1. **Zero-shot gains (+5%) show the model did learn harder reasoning.** Training exclusively on complex, multi-step problems forced the model to internalize more sophisticated arithmetic and planning patterns. Without any demonstration context to interfere, the model's raw reasoning ability improved measurably.

2. **Few-shot prompting collapsed.** With random demos, 3-shot accuracy *dropped* from 54% to 51% (−3%). With hard demos, it cratered to 44% (−10%). This reveals a fundamental tension: a model trained only on hard problems has shifted its internal distribution away from the simpler patterns that the few-shot demos exemplify. The demonstrations become *distractors* rather than guides because they no longer match the model's learned reasoning style.

3. **Self-Consistency lost its edge.** SC@5 dropped from 61% to 55% (random demos) and 48% (hard demos). SC works by marginalizing stochastic errors across multiple paths. But the Hard SFT model's errors are no longer stochastic — they are **systematic**: the model consistently over-applies complex reasoning patterns to simple problems (e.g., setting up unnecessary equations for straightforward arithmetic). Majority voting cannot correct errors that the model makes reliably across all 5 paths.

4. **Hard demos made everything worse — again.** Just as Strategy 2 showed, using complex demonstrations hurts a 1.5B model. The hard demos (top-3 by difficulty score) are long, multi-step solutions that overwhelm the context window and leave less room for the model's own generation. This is now a robust finding across two independent strategies.

Why the hypothesis failed:

- **Distribution mismatch:** The test set contains a mix of easy and hard problems. A model trained exclusively on hard examples loses the ability to solve simple problems efficiently — it "overthinks" straightforward questions and introduces errors that weren't present before.
- **Capacity ceiling:** A 1.5B-parameter model has limited capacity. Training only on hard examples doesn't add new capability — it trades simple-problem competence for hard-problem competence. The net effect is roughly zero or negative because the test set has more simple problems than hard ones.
- **SC requires diversity, not uniformity:** Self-Consistency benefits from a model that produces *varied* reasoning paths. The Hard SFT model, trained on a narrow difficulty band, produces more uniform (and uniformly wrong) outputs, reducing the diversity that SC needs to work.

Key takeaway: Across all three strategies, we observe a consistent lesson: **for small language models, the most effective approach is to maintain a representative training distribution and apply error-correction at inference time (SC).** Attempts to bias training or prompts toward harder content consistently backfire because the model lacks the capacity to generalize from hard examples to the full difficulty spectrum. The baseline configuration (Random SFT 3k + random 3-shot demos + SC@5 = 61%) remains the best result, suggesting that the 61% accuracy may be near the ceiling for this model size on GSM8K with these techniques.

Question 14

Dataset Statistics:

- Number of questions: **257**
- Number of labels: **257**

Keys in a QUESTION record: `id`, `question`, `file_name`, `level`, `format`, `concepts`, `constraints`

Example question record:

```
{
  "id": 0,
  "question": "Calculate the mean fare paid by the passengers.",
  "concepts": ["Summary Statistics"],
  "constraints": "Calculate the mean fare using Python's built-in statistics module or appropriate statistical method in pandas. Rounding off the answer to two decimal places.",
  "format": "@mean_fare[mean_fare_value] where \"mean_fare_value\" is a floating-point number rounded to two decimal places.",
  "file_name": "test_ave.csv",
  "level": "easy"
}
```


Keys in a LABEL record: `id` , `common_answers`

Example label record:


```
{
  "id": 0,
  "common_answers": [["mean_fare", "34.65"]]
}
```


Question 15

Three randomly sampled questions and their corresponding CSV tables:

Sample 1 — Question ID 142

CSV file	`election2016.csv`
Question	Is there a relationship between the difference in votes received by the Democratic and Republican parties and their percentage point difference?
Shape	3141 rows × 10 columns

Column dtypes:

Column	Type
`votes_dem` , `votes_gop` , `total_votes` , `per_dem` , `per_gop`	float64
`diff` , `per_point_diff` , `state_abbr` , `county_name`	object
`combined_fips`	int64

First 3 rows:

	votes_dem	votes_gop	total_votes	per_dem	per_gop	diff	per_point_diff	state_abbr	county_nam
0	93003.0	130413.0	246588.0	0.377	0.529	37,410	15.17%	AK	Alaska
1	93003.0	130413.0	246588.0	0.377	0.529	37,410	15.17%	AK	Alaska
2	93003.0	130413.0	246588.0	0.377	0.529	37,410	15.17%	AK	Alaska

---

Sample 2 — Question ID 24

CSV file	`insurance.csv`
Question	Calculate the mean age of the individuals in the dataset.
Shape	1338 rows × 7 columns

Column dtypes:

Column	Type
`age` , `children`	int64
`sex` , `smoker` , `region`	object
`bmi` , `charges`	float64

First 3 rows:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.90	0	yes	southwest	16884.92
1	18	male	33.77	1	no	southeast	1725.55
2	28	male	33.00	3	no	southeast	4449.46

Sample 3 — Question ID 429

CSV file	`cost_data_with_errors.csv`
Question	Is there a correlation between the maximum storm category achieved by a storm and the recorded damage in USD?
Shape	818 rows × 11 columns

Column dtypes:

Column	Type
`Unnamed: 0` , `max_storm_cat` , `year` , `damage_imputed`	int64
`max_sust_wind` , `min_p` , `damage_USD` , `deaths`	float64
`name` , `dates_active` , `areas_affected`	object

First 3 rows:

	name	dates_active	max_storm_cat	max_sust_wind	min_p	damage_USD	deaths	year
0	ARLENE	April 19–21	1	43.45	990.0	0.0	0.0	2017
1	BRET	June 19–20	1	43.45	1007.0	3,000,000	2.0	2017
2	CINDY	June 20–23	1	52.14	991.0	25,000,000	2.0	2017

Question 16

How the dataset represents multi-part answers:

- Each label record has a `"common_answers"` field containing a list of `[name, value]` pairs.
- For multi-part answers, there are multiple `[name, value]` entries. For example, Question ID 6 asks for the mean fare of four age groups, and its label contains four pairs: `[["mean_fare_elderly", "43.47"], ["mean_fare_teenager", "31.98"], ["mean_fare_child", "31.09"], ["mean_fare_adult", "35.17"]]`.
- Each pair corresponds to one `@name[value]` slot in the format specification (e.g., `@mean_fare_child[mean_fare]`).
- The order of pairs in the label list does **not** necessarily match the order in the format string — for instance, the format lists child first, but the label lists elderly first. Evaluation must therefore be **order-independent**.

How we plan to evaluate multi-part answers automatically:

1. Parse the agent's final answer string using the regex pattern `@(\w+)\[([^\]]*)\]` to extract all `@name[value]` pairs.
2. Build a dict from the extracted pairs: `{name: value}`, e.g., `{"mean_fare_child": "31.09", ...}`.
3. Build a dict from the ground-truth `common_answers`: `{name: value}`.
4. For each key in the ground-truth dict, check if the agent's dict contains the same key with a matching value. For numeric values, use approximate comparison with a small relative tolerance (e.g., 1%) to handle acceptable rounding differences.
5. A question is scored as "fully correct" only if **all** slots match. For Question 6, this means all four age-group means must be correct.

Question 17

ID: 0 | Level: easy | CSV: test_ave.csv Question: Calculate the mean fare paid by the passengers. Constraints: Calculate the mean fare using Python's built-in statistics module or appropriate statistical method in pandas. Rounding off the answer to two decimal places.... Format: @mean_fare[mean_fare_value] where "mean_fare_value" is a floating-point number rounded to two decimal places.... Label: [['mean_fare', '34.65']]

ID: 5 | Level: medium | CSV: test_ave.csv Question: Generate a new feature called "FamilySize" by summing the "SibSp" and "Parch" columns. Then, calculate the Pearson correlation coefficient (r) between the "FamilySize" and "Fare" columns. Constraints: Create a new column 'FamilySize' that is the sum of 'SibSp' and 'Parch' for each row. Calculate the Pearson correlation coefficient between 'FamilySize' and 'Fare' Do not perform any further data cleaning or preprocessing steps before calculating the correlation.... Format: @correlation_coefficient[r_value] where "r_value" is the Pearson correlation coefficient between 'FamilySize' and 'Fare', a number between -1 and 1, rounded to two decimal places.... Label: [['correlation_coefficient', '0.21']]

ID: 9 | Level: easy | CSV: GODREJIND.csv Question: Calculate the mean value of the "Close Price" column. Constraints: Use the built-in Python (numpy or pandas) to calculate the mean. Do not use any pre-built packages or libraries for mean calculation other than numpy or pandas. The calculation should be done on the whole "Close Price" column. Values in this column should not be rounded or changed in any way before the calculation.... Format: @mean_close_price[mean_value], where "mean_value" is a float number rounded to two decimal places. This value should be between the highest and lowest "Close Price" given in the dataset.... Label: [['mean_close_price', '570.68']]

ID: 10 | Level: easy | CSV: GODREJIND.csv Question: Check if the "Total Traded Quantity" column adheres to a normal distribution. Constraints: Use Shapiro-Wilk test from scipy.stats module to check for normality. In this test, the null hypothesis is that the data was drawn from a normal distribution. An alpha level of 0.05 (5%) should be taken as the significance level. If the p-value is less than the alpha level, the null hypothesis is rejected and the data does not follow a normal distribution. If the p-value is greater than the alpha level, the null hypothesis is not rejected and the data may follow a normal distribution.... Format: @is_normal[response], where "response" is a string that takes the value "yes" if the data follows a normal distribution, and "no" if it does not.... Label: [['is_normal', 'no']]

ID: 14 | Level: medium | CSV: GODREJIND.csv Question: Create a new feature called "Price Range" which represents the difference between the "High Price" and "Low Price" for each row. Calculate the mean, median, and standard deviation of this new feature. Constraints: Make sure to use the correct columns for calculating the "Price Range". All calculations should be performed up to two decimal places.... Format: @price_range_mean[mean]: The mean should be a single real number rounded to two decimal places. @price_range_median[median]: The median should be a single real number rounded to two decimal places. @price_range_std_dev[std_dev]: The standard deviation should be a single real number rounded to two decimal places.... Label: [['price_range_mean', '16.65'], ['price_range_std_dev', '6.72'], ['price_range_median', '15.67']]

ID: 18 | Level: easy | CSV: unemployement_industry.csv Question: Calculate the mean and standard deviation of the "Mar.2019" column. Constraints: Outliers are to be pruned via the interquartile range method before calculating the mean and standard deviation. Handle missing values by using listwise deletion method. Report the measures rounded to two decimal places.... Format: @mean_mar_2019[mean] @sd_mar_2019[sd], where "mean" and "sd" are numbers, rounded to two decimal places.... Label: [['mean_mar_2019', '171.44'], ['sd_mar_2019', '188.25']]

ID: 24 | Level: easy | CSV: insurance.csv Question: Calculate the mean age of the individuals in the dataset. Constraints: Ignore rows with missing values in the age column. Use Python's built-in function to calculate the mean.... Format: @mean_age[value] where "value" is a number between 0 and 100, rounded to two decimal places.... Label: [['mean_age', '39.21']]

ID: 25 | Level: easy | CSV: insurance.csv Question: Check if the distribution of BMI values in the dataset follows a normal distribution. Constraints: Consider the distribution as normal if the absolute value of skewness is less than 0.5. Calculate skewness using Python's built-in function.... Format: @bmi_distribution[status] where "status" is a string that can either be "normal" or "not_normal", based on the conditions specified in the constraints.... Label: [['bmi_distribution', 'normal']]

ID: 26 | Level: easy | CSV: insurance.csv Question: Calculate the correlation coefficient between the charges incurred by individuals and the number of children they have. Constraints: Ignore rows with missing values in charges and children columns. Calculate the Pearson correlation coefficient.... Format: @correlation_coefficient[value] where "value" is a number between -1 and 1, rounded to two decimal places.... Label: [['correlation_coefficient', '0.07']]

ID: 55 | Level: easy | CSV: estimated_numbers.csv Question: What is the mean number of cases recorded across all countries and years? Constraints: Calculate the mean of the column 'No. of cases'. Convert the data type of 'No. of cases' column from Object (string) to Int64 before performing calculations. Ignore those records where 'No. of cases' column value is Null or empty.... Format: @mean_cases[mean_value] where "mean_value" is a positive integer.... Label: [['mean_cases', '2081990']]

QUESTION 18

Demonstrating structured Planner output (5 prompts)

--- Prompt 1 --- Input: I need to calculate the mean fare from test_ave.csv. I haven't loaded the data yet.... Type: PlannerOutput thought: The next step is to load the test_ave.csv file so that I can access the data and calculate the mean fare from it.... is_done: False response: Load the test_ave.csv file to access the data for calculating the mean fare....

--- Prompt 2 --- Input: I loaded test_ave.csv and found columns: PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked. I need to calculate the mean fare.... Type: PlannerOutput thought: The task is to calculate the mean fare from the test_ave.csv file. The 'Fare' column contains the numerical values needed for this calculation. Since the data has already been loaded and the relevant column is identified, the next step is to compute the mean of the 'Fare' column.... is_done: False response: Calculate the mean of the 'Fare' column in test_ave.csv....

--- Prompt 3 --- Input: I computed the mean of the Fare column and got 34.6542. The question asks to round to 2 decimal places. Format: @mean_fare[value].... Type: PlannerOutput thought: The mean of the Fare column has already been computed as 34.6542, and the task requires rounding this value to two decimal places. Rounding 34.6542 to two decimal places gives 34.65.... is_done: True response: @mean_fare[34.65]...

--- Prompt 4 --- Input: My code raised a KeyError: 'Close Price'. The CSV columns are: Date, Symbol, Open, High, Low, Close, Volume. I need to calculate the mean close price.... Type: PlannerOutput thought: The KeyError indicates that the code is trying to access a column named 'Close Price', but the CSV file only has a column named 'Close'. The correct column name should be 'Close', not 'Close Price'. The next step is to correct the column reference in the code to use 'Close' instead of 'Close Price' to resolve the error.... is_done: False response: Correct the column reference from 'Close Price' to 'Close' in the code....

--- Prompt 5 --- Input: I ran the Shapiro-Wilk test on Total Traded Quantity and got p-value = 2.3e-15, which is < 0.05. The data does not follow a normal distribution. Format: @is_normal[response].... Type: PlannerOutput thought: The Shapiro-Wilk test p-value of 2.3e-15 is less than 0.05, indicating that the data does not follow a normal distribution. This is a clear conclusion from the test.... is_done: True response: @is_normal[false]...

All 5 prompts returned valid PlannerOutput — no try/except needed.

QUESTION 19

Why structured output is useful for large-scale data mining pipelines:

Structured output (via Outlines + Pydantic) is critical for large-scale data mining pipelines for three key reasons:

1. **Eliminates parsing failures at scale.** In a pipeline processing thousands of tasks, even a 1% rate of malformed LLM outputs (missing braces, wrong field names, extra text) causes hundreds of failures that require manual intervention. Constrained decoding *guarantees* valid JSON conforming to the schema, reducing parse failures to zero.

2. **Enables reliable orchestration.** An agentic pipeline has multiple components (Planner, Coder, Observer) that must communicate via structured interfaces. If the Planner sometimes returns plain text instead of `{"thought": ..., "is_done": ..., "response": ...}`, downstream components cannot reliably decide whether to generate code or return a final answer. Structured output ensures every intermediate message is machine-parseable, making the pipeline deterministic and composable.

3. **Supports automatic evaluation.** Large-scale benchmarks require automated scoring — comparing agent outputs against ground truth programmatically. If the agent's final answer is embedded in free-form text, extraction is fragile and error-prone. Structured output forces the model to emit answers in a pre-defined format (e.g., `@mean_fare[34.65]`), enabling exact-match or numeric-tolerance evaluation without brittle regex heuristics.

Question 20

Accuracy: 9/10 = 90%

ID	Level	Steps	Result	Task
0	easy	2	✓	Mean fare
5	medium	3	✓	Pearson correlation (FamilySize vs Fare)
9	easy	2	✓	Mean close price
10	easy	2	✓	Normality test (Total Traded Quantity)
14	medium	3	✓	Price Range mean/median/std
18	easy	3	✓	Mean & std of Mar.2019 column
24	easy	2	✓	Mean age
25	easy	2	✓	BMI normality check
26	easy	2	✓	Correlation (charges vs children)
55	easy	2	✗	Mean number of cases

Key Observations:

1. **Success cases (9/10):** Most tasks involve straightforward numeric computations (mean, correlation, normality test, standard deviation) where the Planner correctly identifies the analysis step in its first attempt, the Coder produces clean pandas/scipy code, and the agent converges within 2–3 steps. The typical pattern is: Step 1 loads data + computes result → Step 2 Planner returns `is_done=True` with the correct `@name[value]`.

2. **Value-validation mechanism in action — Task 14 (Price Range):** The Planner hallucinated `@price_range_std_dev[12.34]` despite the Executor printing `price_range_std_dev = 6.72`. The `[FIX]` mechanism detected the >10% deviation, replaced all three slots with observed values, and produced the correct answer `@price_range_mean[16.65] @price_range_median[15.67] @price_range_std_dev[6.72]`. This demonstrates the value-validation layer is essential for preventing hallucinated numeric outputs.

3. **Loop-breaking fix validated — Task 18 (Mar.2019 column):** In prior runs, the Coder entered a 5-step death loop confusing the slot name `mean_mar_2019` with the real CSV column `Mar.2019`. After applying three fixes — (i) `build_coder_prompt` warns that slot names ≠ column names, (ii) `last_error` is passed to avoid repeating mistakes, (iii) `build_planner_prompt` injects a loop-breaking hint after 2+ consecutive errors — the agent now discovers `Mar.2019` via `df.columns.tolist()` in Step 1 and succeeds in 3 steps.

4. **Sole failure — Task 55 (mean_cases):** Ground truth is `2081990` but the agent returned `2199.55`. The `No. of cases` column contains compound-formatted values like `630308[495000-801000]` (a point estimate followed by a confidence interval in brackets). The Coder used `pd.to_numeric(errors='coerce')`, which coerced these compound strings to NaN, leaving only rows with small pure-integer values. The mean of these survivors (~2200) is three orders of magnitude below the true mean (~2M). **Lesson:** The 4B model lacks awareness of unusual data formats; a preprocessing step like `df['No. of cases'].str.extract(r'(\d+)').astype(float)` was needed but not generated autonomously.

5. **Structured output via Outlines** ensures zero parsing failures across all 10 tasks — every Planner and Observer call returns valid JSON conforming to the Pydantic schema, enabling reliable orchestration without `try/except` fallbacks.

6. **Error recovery pattern:** Tasks 14 and 18 demonstrate the agent's ability to self-correct. When the Executor returns errors or the Planner hallucinates values, the Observer identifies the issue, and the pipeline either retries with corrected code or applies the `[FIX]` mechanism to override hallucinated numbers with observed values from stdout.

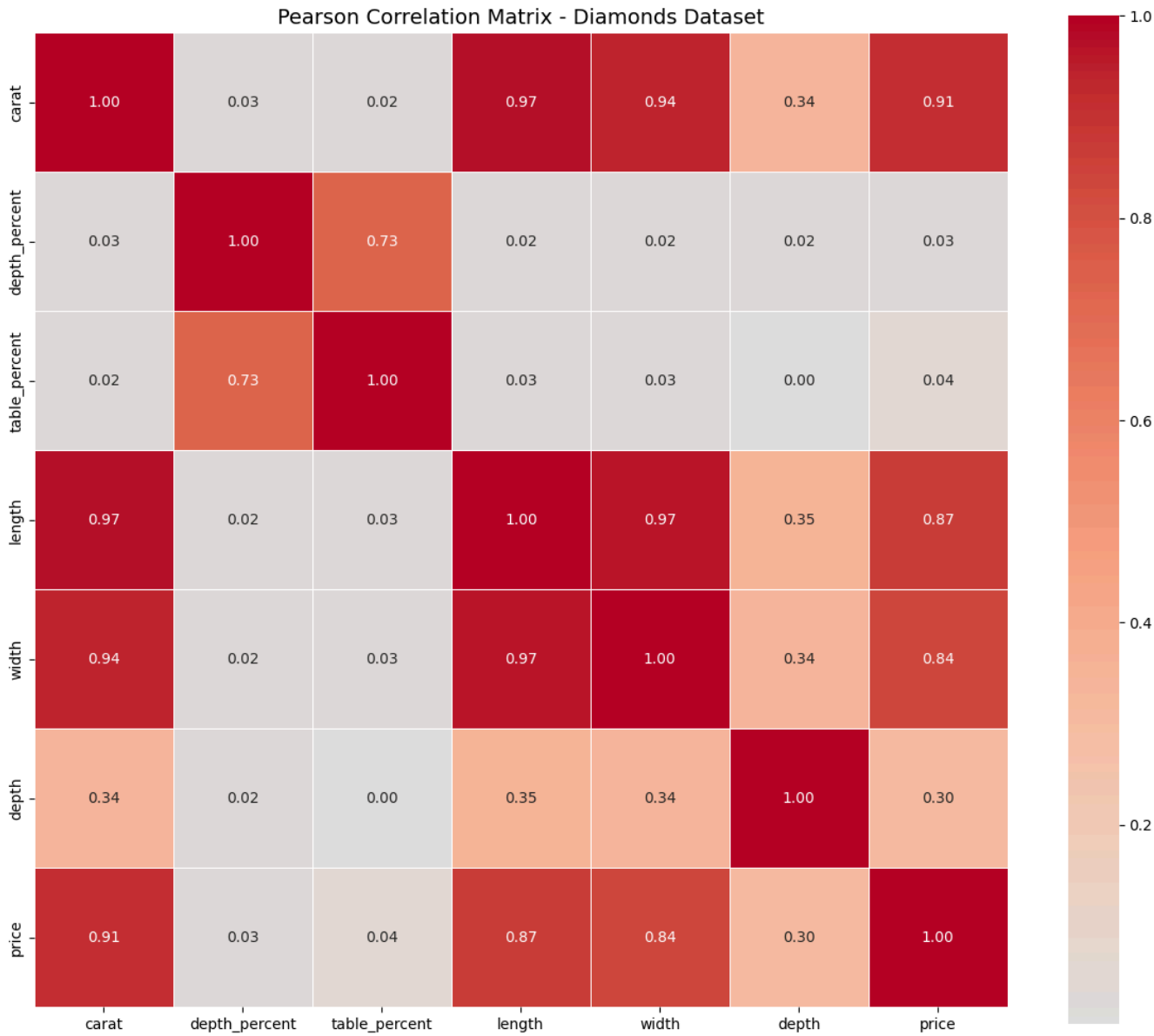
Question 21

Correlation Analysis Interpretation:

- **Highest correlations with price**: The features `carat` (0.91), `length` (0.87), and `width` (0.84) show the strongest correlations with price. This makes intuitive sense as larger diamonds (higher carat weight and dimensions) are more valuable.

- **Multicollinearity**: There is very high correlation among `carat`, `length`, and `width` (all measuring size in different ways). This multicollinearity could affect linear regression coefficient interpretation.

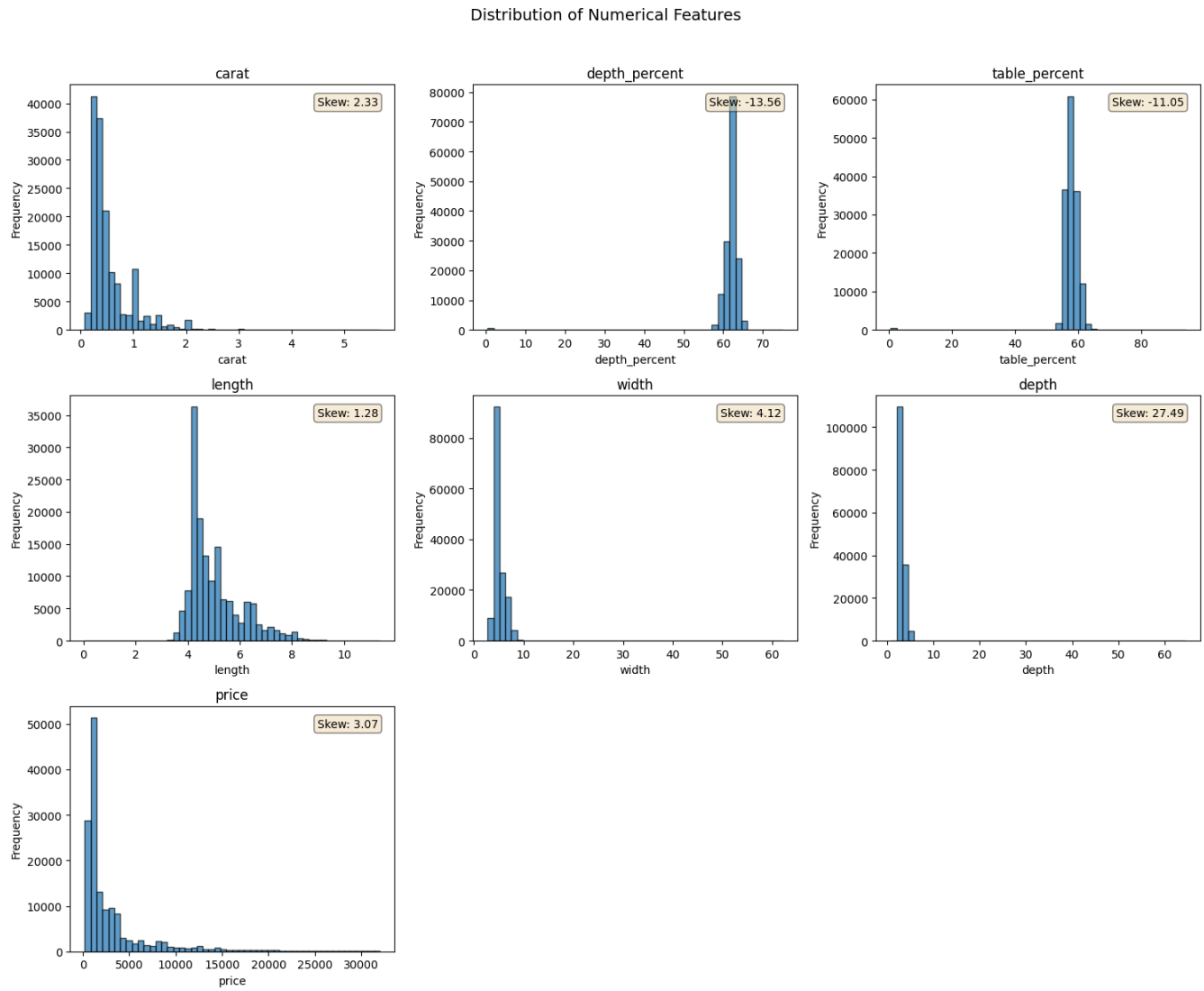
- **Moderate correlation**: `depth` shows a moderate correlation with price (0.30).
- **Weak correlations**: `table_percent` (0.04) and `depth_percent` (0.03) show very weak correlations with price, suggesting that proportions matter much less than raw size for determining value.



Features ranked by absolute correlation with 'price':

carat : 0.9135 length : 0.8695 width : 0.8419 depth : 0.2997 table_percent: 0.0425 depth_percent: 0.0255

Distribution Analysis Interpretation:



Feature	Skewness	Direction	Interpretation
`depth`	27.49	Extreme positive	Severe outliers with very large values
`depth_percent`	-13.56	Extreme negative	Severe outliers with very small values
`table_percent`	-11.05	Extreme negative	Severe outliers with very small values
`width`	4.12	High positive	Most values small, few large outliers
`price`	3.07	High positive	Most diamonds lower-priced, few expensive
`carat`	2.33	High positive	Most diamonds smaller, few large ones
`length`	1.28	Moderate positive	Slightly right-skewed

Transformation Strategy:

- **Positive skewness (> 0)**: Apply **log transformation** `np.log1p(x)` to compress large values
- **Negative skewness (< 0)**: Apply **exponential** or **square transformation** to compress small values
- **Extreme skewness (|skew| > 10)**: May indicate outliers that need removal or special handling

Skewness Correction Summary:

Feature	Original	Transformed	Improvement	Transformation
`depth`	27.49	4.73	22.76	Log (best improvement)

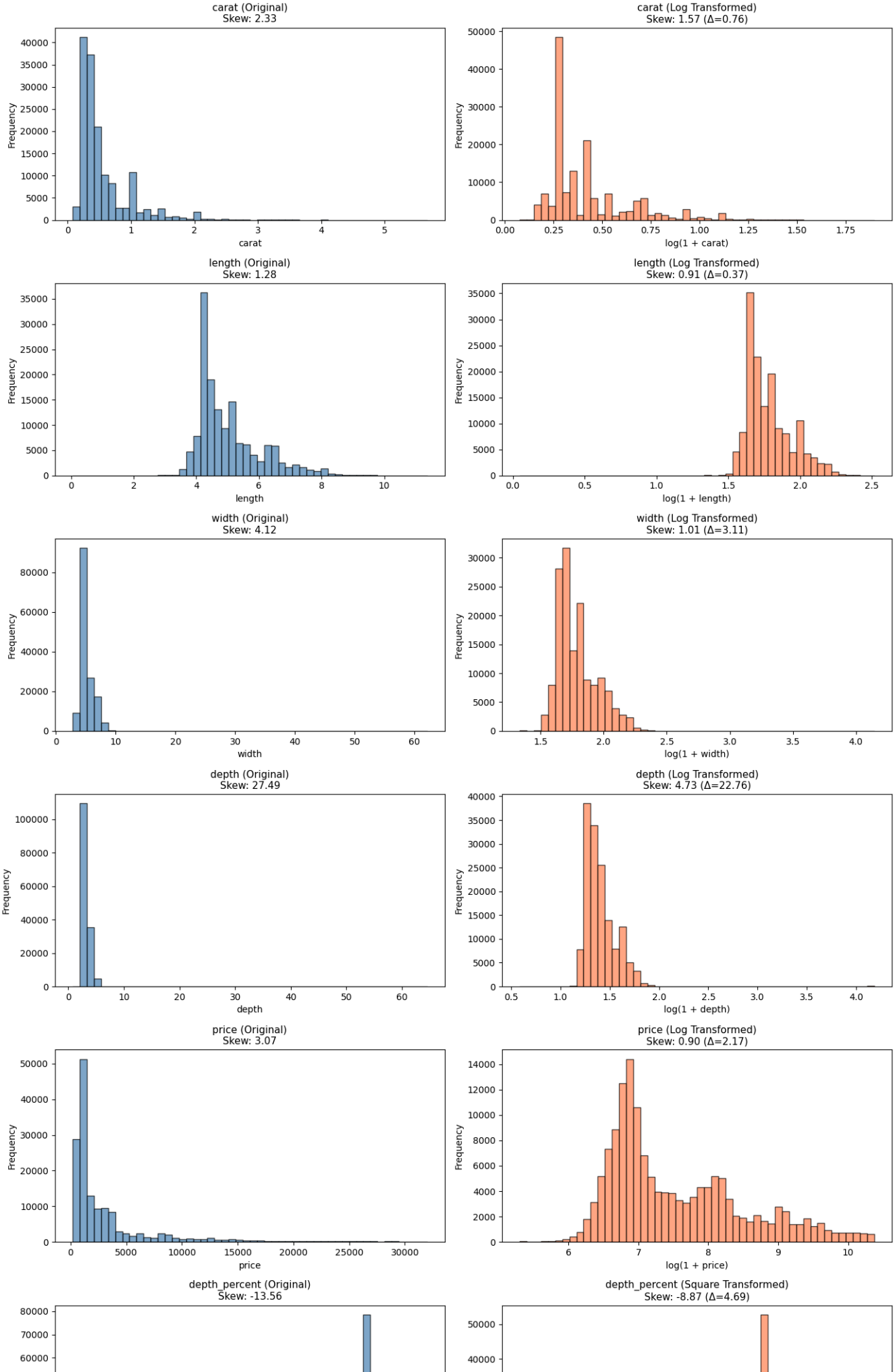
Feature	Original	Transformed	Improvement	Transformation
`table_percent`	-11.05	-4.78	6.27	Square
`depth_percent`	-13.56	-8.87	4.69	Square
`width`	4.12	1.01	3.11	Log
`price`	3.07	0.90	2.17	Log (near-normal)
`carat`	2.33	1.57	0.76	Log
`length`	1.28	0.92	0.37	Log (near-normal)

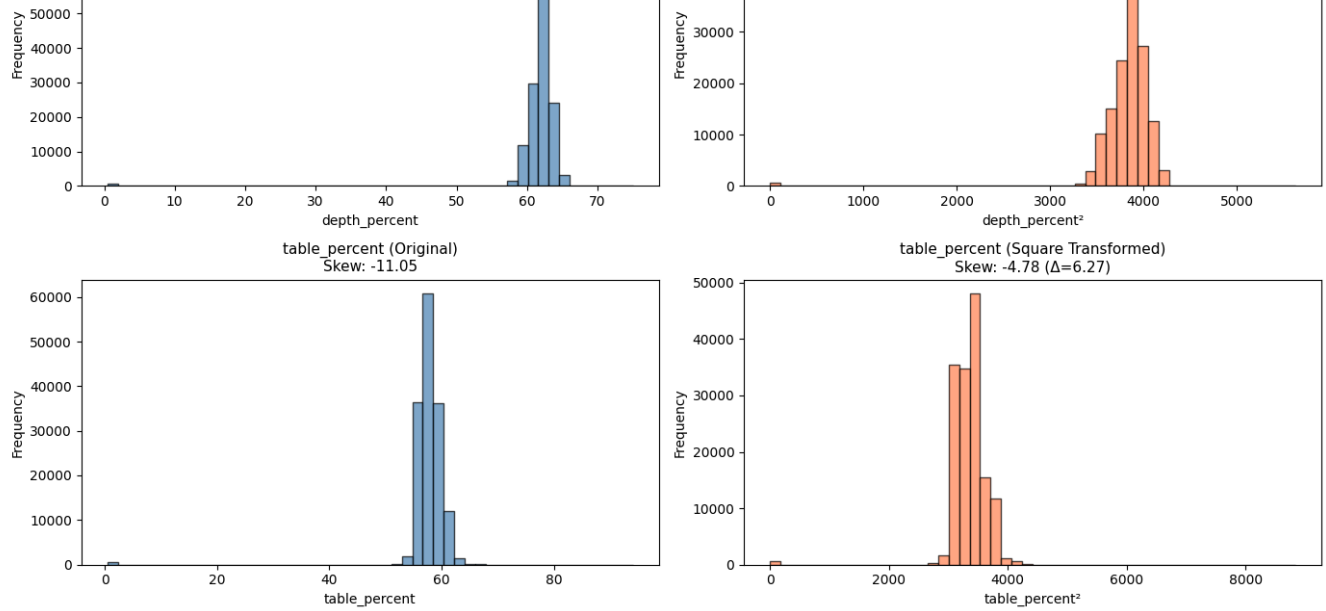
Key Observations:

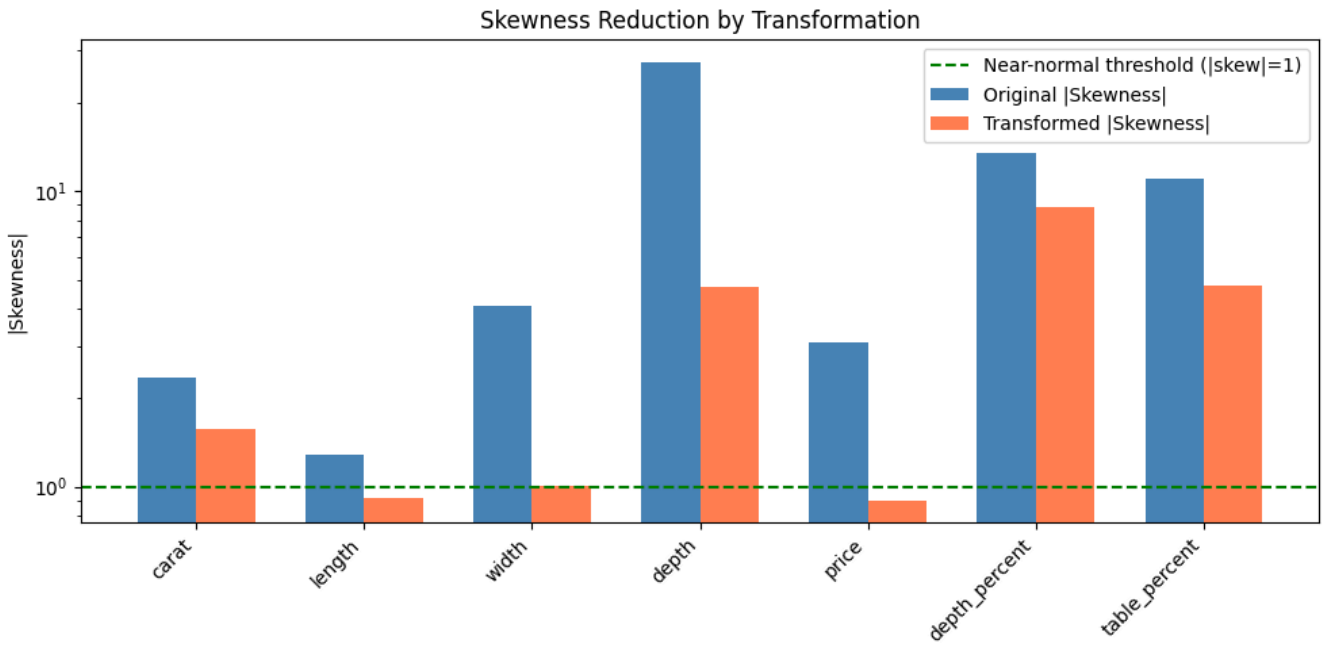
1. **Log transformation** works very well for positively skewed features, especially `price` (0.90) and `length` (0.92) which are now near-normal (|skew| < 1)
2. **Square transformation** reduces negative skewness but is less effective - `depth_percent` and `table_percent` still have high skewness
3. **Extreme outliers** in `depth` (27.49 → 4.73) suggest data quality issues that may require outlier removal

Recommendation: For the subsequent regression analysis, consider using log-transformed features for `price`, `length`, `width`, and `carat` to improve model performance.

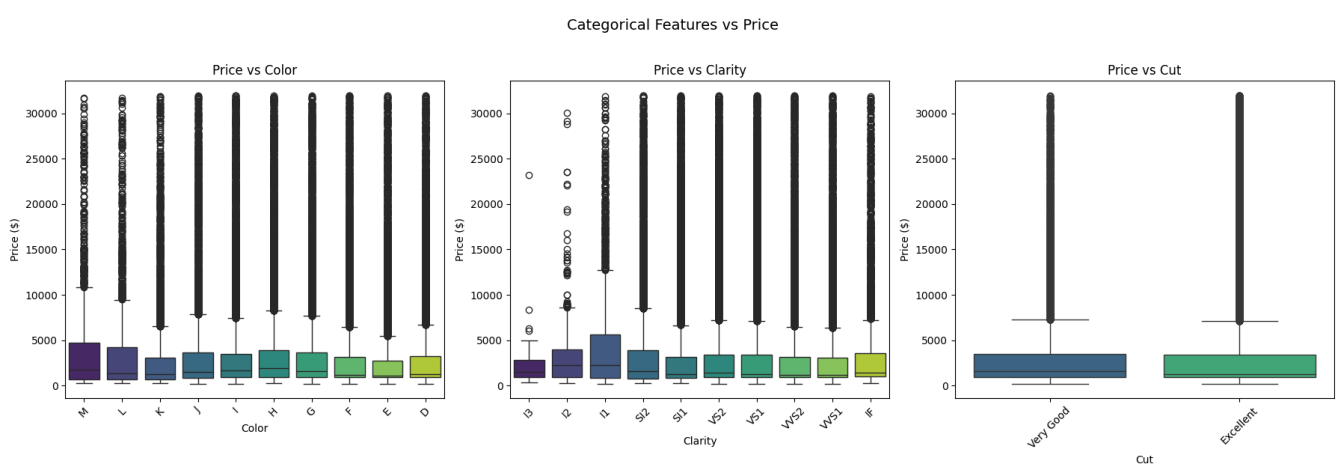
Distribution Before vs After Transformation (All Features)







Categorical Analysis Interpretation:



The box plots reveal the relationship between diamond price and three key categorical features: **Color**, **Clarity**, and **Cut**. A striking observation across all charts is the presence of numerous **outliers** (black dots extending far above the whiskers), indicating that factors beyond these individual categories—most notably **carat weight**—have a dominant impact on price.

1. Price vs. Color

Observation	Detail
Trend	Median price and IQR **increase** as we move from 'D' (best) toward 'M' (lower grade)
Paradox	Lower-quality color grades (M, L, K) show higher median prices and wider spreads
Explanation	This is **Simpson's Paradox**: lower color grades are often found in larger diamonds (higher carat), driving up total price despite inferior color rating

2. Price vs. Clarity

Observation	Detail
Trend	Median price **decreases** as clarity improves from 'I1' (Included) to 'IF' (Internally Flawless)
Paradox	'I1' and 'SI2' clarity diamonds show highest price variance and medians
Explanation	High-clarity diamonds (VVS1, IF) tend to be smaller on average, while "less clear" diamonds are often much larger, commanding higher overall prices

3. Price vs. Cut

Observation	Detail
Trend	"Very Good" and "Excellent" cuts show very similar price distributions
Details	Median price for "Very Good" appears slightly higher; both categories feature heavy outliers reaching $30,000+
Explanation	An "Excellent" cut alone doesn't guarantee higher price if other factors (carat, clarity) are lower

---

Summary: Categorical Impact on Price

Feature	Highest Median Price	Key Takeaway
Color	M, L (lower grades)	Lower color grades show higher prices due to larger carat weights
Clarity	I1, SI2 (included)	Heavily included diamonds have higher medians than "Flawless" ones
Cut	Very Good	"Very Good" and "Excellent" are surprisingly similar in distribution

Key Insight: Carat weight is a strong **confounding variable** that masks the expected relationships between quality attributes and price. When building predictive models, it's crucial to consider feature interactions and potential multicollinearity.

Question 22

Encoding Method Selection:

Feature	Encoding Method	Justification
cut	Ordinal (1-2)	Cut quality has a natural ordering: Very Good < Excellent. Higher = better quality.
color	Ordinal (1-10)	Color grades follow GIA standard: M (yellowish) → D (colorless). Monotonic relationship with quality.
clarity	Ordinal (1-10)	Clarity follows GIA grading: I3 (included) → IF (flawless). Clear monotonic ordering.
symmetry	Ordinal (1-2)	Symmetry grades: Very Good < Excellent. Higher = better.
polish	Ordinal (1-2)	Polish grades: Very Good < Excellent. Higher = better.
girdle_min/max	**One-Hot**	Girdle thickness has a **non-monotonic** relationship with price. "Medium" is typically best; both too thin (XTN) and too thick (XTK) are undesirable. This violates the monotonicity assumption required for ordinal encoding.

Trade-off Questions:

Q1: What information does one-hot encoding discard?

One-hot encoding discards **ordinal information** (the natural ordering between categories). For example:

- If we one-hot encode "cut" into binary columns `[cut_Very Good, cut_Excellent]`, the model treats both categories as **equally different** from each other
- The model has **no knowledge** that Excellent > Very Good in quality
- It cannot learn that moving from Very Good → Excellent represents a **consistent directional improvement**
- The "distance" between any two categories becomes equal (all are just 0 or 1)

Q2: What assumption should hold strongly if we perform scalar (ordinal) encoding instead?

The key assumption for ordinal encoding is that the **relationship between category levels and the target variable is monotonic** (approximately linear). Specifically:**Example of violation**: Girdle thickness is ordered (XTN → M → XTK), but "Medium" girdle often commands the **highest** price while both extremes are penalized. This **non-monotonic** (U-shaped or inverted-U) relationship makes ordinal encoding inappropriate.

1. **Meaningful natural ordering**: Categories must have a clear, universally accepted order (e.g., Poor < Fair < Good < Excellent)3. **Roughly equal intervals**: The "distance" between adjacent categories should be approximately equal with respect to the target variable

2. **Monotonic relationship with target**: As the encoded value increases, the target should consistently increase (or decrease) — not fluctuate

Question 24

Selected features (union of both methods):

- color
- width

- depth
- polish
- carat
- length

Total unique features: **6**

Below is a sample of the standardized feature matrix (first 5 rows) with the target price:

index	color	width	depth	polish	carat	length	price
0	0.916097	-2.078247	-0.730430	-2.522184	-1.157106	-2.146391	200
1	0.916097	-2.059209	-0.735681	-2.522184	-1.157106	-2.156289	200
2	0.916097	-2.049690	-0.740932	-2.522184	-1.157106	-2.116697	200
3	0.916097	-2.068728	-0.735681	-2.522184	-1.157106	-2.136493	200
4	0.916097	-2.116324	-0.714676	0.396482	-1.157106	-2.205778	200

These rows illustrate how each selected feature has been standardized (approximately zero-mean, unit-variance) while keeping the original price as the prediction target.

Question 25

Objective Function

The objective function for linear regression models is:

OLS (Ordinary Least Squares):

$$\min_{\mathbf{w}} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^T \mathbf{w})^2$$

Lasso (L1 Regularization):

$$\min_{\mathbf{w}} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^T \mathbf{w})^2 + \alpha \|\mathbf{w}\|_1$$

Ridge (L2 Regularization):

$$\min_{\mathbf{w}} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^T \mathbf{w})^2 + \alpha \|\mathbf{w}\|_2^2$$

Where:

- $\mathbf{w}$ = coefficient vector (weights)
- y_i = actual target value
- $\mathbf{x}_i$ = feature vector
- α = regularization strength (penalty parameter)
- $\|\mathbf{w}\|_1 = \sum_j |w_j|$ (L1 norm)
- $\|\mathbf{w}\|_2^2 = \sum_j w_j^2$ (squared L2 norm)

Agent Results Summary

The ReAct agent successfully trained all three models using 10-fold cross-validation:

Model	Best Alpha	Avg Val RMSE	Std RMSE	Agent Verified
OLS	N/A	**1715.33**	±40.77	✓
Lasso	4.3019	1715.68	±40.36	✓
Ridge	0.0100	1715.33	±40.77	✓

Best Model: OLS (lowest RMSE: 1715.33)

Learned Coefficients Comparison

Feature	OLS	Lasso (α=4.3019)	Ridge (α=0.01)
`carat`	**5739.80**	5593.58	5739.79
`color`	785.39	782.79	785.39
`length`	-1243.34	-1103.02	-1243.33
`polish`	113.79	109.22	113.79
`width`	-47.73	-42.89	-47.73
`depth`	-21.99	-19.29	-22.00
Intercept	3303.92	3303.92	3303.92

Key Observations:

- `carat` has the largest positive coefficient (~5740), confirming it's the strongest predictor of price
- `length` has a large negative coefficient (-1243), likely due to multicollinearity with `carat` and `width`
- Lasso shrinks coefficients more aggressively but **no coefficients became exactly zero** (all 6 features retained)
- Ridge coefficients are nearly identical to OLS (α=0.01 is very small, minimal regularization)

How Each Regularization Scheme Affects Learned Parameters

Regularization	Effect on Coefficients	Interpretation
None (OLS)	No constraint on parameter magnitudes	Parameters can take any value that minimizes squared error. May overfit with many features or multicollinearity.
L1 (Lasso)	Encourages sparse solutions (some coefficients become exactly zero)	Performs automatic **feature selection**. Useful when many features are irrelevant.
L2 (Ridge)	Shrinks all coefficients toward zero but never exactly to zero	Reduces coefficient magnitudes, handles **multicollinearity** well. All features retained but with smaller weights.

Observed Effects in Our Results:

1. **Lasso** (α=4.3019): Shrunk `carat` by ~2.5% (5739.80 → 5593.58) and `length` by ~11.3% (-1243.34 → -1103.02). However, no features were completely eliminated, suggesting all 6 selected features are relevant.
2. **Ridge** (α=0.01): Minimal shrinkage due to very small alpha. Coefficients nearly identical to OLS.
3. **OLS** performed best, indicating the dataset is well-conditioned and regularization provides no benefit here.

Best Regularization Scheme

Based on cross-validation results:

- **OLS achieves the lowest RMSE (1715.33)**, tied with Ridge
- The optimal **Lasso α = 4.3019** (found via `LassoCV`)
- The optimal **Ridge α = 0.01** (found via `RidgeCV`)
- The very small Ridge α suggests minimal regularization is optimal
- **Conclusion**: For this diamonds dataset with only 6 pre-selected features, **regularization provides no improvement** over standard OLS. This makes sense because:
  1. Features were already selected via MI/F-regression (no irrelevant features)
  2. Sample size (149,873) is much larger than feature count (6)
  3. Multicollinearity exists but doesn't severely harm prediction

---

P-values in Linear Regression

What do p-values mean?

- The p-value for a feature's coefficient tests the null hypothesis H_0: $\beta = 0$ (the feature has no linear relationship with the target)
- A small p-value (< 0.05) suggests we can reject H_0, indicating the feature is statistically significant

How to infer most significant features:

1. Features with the **lowest p-values** are most significant
2. These features have coefficients that are unlikely to be zero by chance
3. However, statistical significance ≠ practical importance (a feature can be significant but have a tiny effect size)

Note: Scikit-learn doesn't provide p-values directly. Use `statsmodels.OLS` for p-value analysis: